

PRACTICAL – 2

AIM: - Implementation to Convert the text to word count vectors with ScikitLearn (CountVectorizer).

Theory :

CountVectorizer is a feature extraction technique in NLP that converts text into numerical word count vectors.

It creates a **vocabulary** of all unique words present in the given text corpus.

It generates a **Document-Term Matrix (DTM)**, where rows represent documents, columns represent words, and values represent word frequencies.

By default, it extracts **unigrams (single words)**, while `ngram_range=(2,2)` is used to extract **bigrams (pairs of consecutive words)**.

CountVectorizer is commonly used in **text classification, sentiment analysis, spam detection, and other machine learning applications**.

CODE: -

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 corpus = [
3     'This is the first document.',
4     'This document is the second document.',
5     'And this is the third one.',
6     'Is this the first document?',
7 ]
8 vectorizer = CountVectorizer()
9 X = vectorizer.fit_transform(corpus)
10 vectorizer.get_feature_names_out()
11 print(X.toarray())
12
13 vectorizer2 = CountVectorizer(analyzer='word', ngram_range=(2, 2))
14 X2 = vectorizer2.fit_transform(corpus)
15 vectorizer2.get_feature_names_out()
16
17 print('Vocabulary for vectorizer (unigrams):')
18 display(vectorizer.vocabulary_)
19
20 print('\nVocabulary for vectorizer2 (bigrams):')
21 display(vectorizer2.vocabulary_)
```

OUTPUT : -

```
... [[0 1 1 1 0 0 1 0 1]
      [0 2 0 1 0 1 1 0 1]
      [1 0 0 1 1 0 1 1 1]
      [0 1 1 1 0 0 1 0 1]]
Vocabulary for vectorizer (unigrams):
{'this': 8,
 'is': 3,
 'the': 6,
 'first': 2,
 'document': 1,
 'second': 5,
 'and': 0,
 'third': 7,
 'one': 4}

Vocabulary for vectorizer2 (bigrams):
{'this is': 11,
 'is the': 3,
 'the first': 6,
 'first document': 2,
 'this document': 10,
 'document is': 1,
 'the second': 7,
 'second document': 5,
 'and this': 0,
 'the third': 8,
 'third one': 9,
 'is this': 4,
 'this the': 12}
```



Parul University

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

DEEP LEARNING WITH NLP LABORATORY (203105477)

7th SEMESTER

COMPUTER SCIENCE & ENGINEERING DEPARTMENT

LABORATORY MANUAL

CERTIFICATE

This is to certify that Mr **S Bharat Kumar Varm** a
enrolment no. 2303031241218 has successfully completed his
laboratory experiments in the DEEP LEARNING WITH NLP LABORATORY
(303105480) from the department of CSE-AI during the academic
Year 2026-27



Date of Submission:.....

Staff Incharge:.....

Head of Department:.....

TABLE OF CONTENT

Sr. No	Experiment Title	Page No		Date of Start	Date of Completion	Sign	Marks (Out of 10)
		From	To				
1.	Implementation of preprocessing of Text with NLTK (Tokenization, Stemming, Lemmatization and removal of stopwords in NLP).						
2.	Implementation to Convert the text to word count vectors with ScikitLearn (Count Vectorizer).						
3.	Implementation to Convert the text to word frequency vectors with ScikitLearn (Tfidf Vectorizer).						
4.	Implementation to Convert the text to unique integers with ScikitLearn (Hashing Vectorizer).						
5.	Use the Keras deep learning library and split words with (text_to_word_sequence).						
6.	Use the Keras deep learning library and write a code for encoding with (one_hot).						
7.	Use the Keras deep learning library and write a code for Hash Encoding with (hashing_trick).						
8.	Use the Keras deep learning library give a demo of Tokenizer API.						
9.	Perform an experiment to do sentimental analysis on any of the dataset (like twitter tweets, movie review etc.).						
10.	Perform an experiment using Gensim python library for Word2Vec Embedding.						

PRACTICAL-1

AIM:- Implementation of preprocessing of Text with NLTK (Tokenization, Stemming, Lemmatization and removal of stop words in NLP).

TOKENIZATION:-

Tokenization refers to break down the text into smaller units. It entails splitting paragraphs into sentences and sentences into words. It is one of the initial steps of any NLP preprocessing.

CODE:-

```
text="I am charan i love coding." tokens =  
text.split()  
print(tokens)
```

OUTPUT:-

```
import nltk  
from nltk.tokenize import sent_tokenize, word_tokenize  
  
example_string = """  
I am charan.i love coding.  
"""  
|  
sentences = sent_tokenize(example_string)  
print("Sentences:")  
print(sentences)  
  
words = word_tokenize(example_string)  
print("\nWords:")  
print(words)  
... Sentences:  
['\nI am charan.i love coding.']  
  
Words:  
['I', 'am', 'charan.i', 'love', 'coding', '.']  
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data] Package punkt is already up-to-date!  
[nltk_data] Downloading package punkt_tab to /root/nltk_data...  
[nltk_data] Package punkt_tab is already up-to-date!
```

STEMMING:-

Stemming generates the base word from the word by removing the affixes of the word. It must be noted that stemmers might not always result in semantically meaningful base words.

CODE:-

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["playing", "played", "plays", "happily", "studies"]

for word in words:
    print(word, " --->", stemmer.stem(word))
```

OUTPUT:-



```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["playing", "played", "plays", "happily", "studies"]

for word in words:
    print(word, "---->", stemmer.stem(word))

... playing ----> play
   played ----> play
   plays ----> play
   happily ----> happili
   studies ----> studi
```

LEMMATIZATION: -

Lemmatization involves grouping together the inflected forms of the same word. This way, we can reach out to the base form of any word which will be meaningful in nature. The base form here is called the Lemma.

CODE:-

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

#Download required resources (run once)
nltk.download('wordnet')
nltk.download('punkt')

lemmatizer = WordNetLemmatizer()

sentence = "The cats are running faster than the dogs"

words = word_tokenize(sentence)

for word in words:
    print(word, " ---> ", lemmatizer.lemmatize(word))
```

OUTPUT: -

```
... [nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
The ----> The
cats ----> cat
are ----> are
running ----> running
faster ----> faster
than ----> than
the ----> the
dogs ----> dog
```

REMOVAL OF STOPWORDS: -

Stopword removal is one of the most commonly used preprocessing steps across different NLP applications. The idea is simply removing the words that occur commonly across all the documents in the corpus.

CODE:-

```
import nltk  
  
from nltk.corpus import stopwords  
  
nltk.download('stopwords')  
  
print(stopwords.words('english'))
```

OUTPUT:-



```
Removal of stop words
```

```
import nltk  
from nltk.corpus import stopwords  
nltk.download('stopwords')  
print(stopwords.words('english'))
```

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 's  
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data] Unzipping corpora/stopwords.zip.
```